

Labwork 12.2 - Wordle Game



General Information and Submission Rules:

- By the end of the labwork, you must submit it to Yulearn as **studentID_fullname_labwork12.c**.
- Only use concepts covered in the labs so far.

Q1 - Prepare Input (25p)

- Before any comparison, turn all input to uppercase and validate it: exactly 5 alphabetic characters.

TASKS:

- Write `void to_upper(char s[])`: convert every character in `s` to uppercase string using `toupper(char)` (hint - include `<ctype.h>` library to your code)
- Write `int is_valid_guess(char guess[])` - return 1 only if length of the string is equal to 5.
- Write `int count_char(char word[], char c)` - count frequency of `c` in `word`. It will be used to handle duplicate letters.
- In `main()`:
 - read a guess,
 - validate it,
 - convert to uppercase letters
 - print "VALID" or "ERROR" (print the reason of the error)

```
Enter guess: hi
Must be exactly 5 letters!
Enter guess: crane
Valid: CRANE

count_char("CRANE", 'E') = 1
count_char("SPEED", 'E') = 2
```

Q2 - Find Green Letters - Exact Match (25p)

- Wordle checks every position for an exact match. If `guess[i] == target[i]`, that position is green (value 2).
- This must be done before yellow-checking to handle duplicates correctly.

TASKS:

- Write `void check_exact(char guess[], char target[], int result[], int matched[])`: loop all 5 letters:
 - If there is a match assign `result[i]` to 2, `matched[i]` to 1,
 - Otherwise leave them 0.
- `matched[]` is a 5-element array tracking target letters that have already been claimed.
- Write `int is_win(int result[])`: return 1 if all 5 results equal 2.
- Test:
 - `guess = "CRANE"`
 - `target = "PLANE"`

```
guess = CRANE
target = PLANE

Position 0 - no match
Position 1 - no match
Position 2 - GREEN
Position 3 - GREEN
Position 4 - GREEN

result[] = {0, 0, 2, 2, 2}
is_win? No
```

Q3 - Yellow Check (25p)

- It also checks unmatched guess letters.
- The `matched[]` array from Q2 prevents the same target letter from scoring twice.
- Then you can use given function to print the result as row:

```
void print_row(char guess[], int result[]) {
    for (int i=0; i<5; i++) {
        char *tag = (result[i]==2) ? "[G]"
                        : (result[i]==1) ? "[Y]" : "[_]";
        printf("%s%c ", tag, guess[i]);
    }
}
```

```

    }
    printf("\n");
}

```

TASKS:

- Write void **check_partial**(char guess[], char target[], int result[], int matched[]) - for each i where result is not equal to 2 scan target if found return 1.
- Write void **score_guess**(char guess[], char target[], int result[]) - calls both check functions in order.
- Print the result by using the given function.

```
target = SCALE //hidden from the user
```

```
Guess: CRANE
```

```
[_]C  [_]R  [Y]A  [_]N  [G]E
```

```
Guess: FLAKE
```

```
[_]F  [_]L  [G]A  [_]K  [G]E
```

```
Guess: SHADE
```

```
[G]S  [_]H  [G]A  [_]D  [G]E
```

```
Guess: SCALE
```

```
[G]S  [G]C  [G]A  [G]L  [G]E
```

Q4 - Play Game (25p)

- Store all guesses and results in 2D arrays so you can reprint the full board on every turn.

TASKS:

- Declare **char history[6][6]** - 6 guesses each 5 chars + '\0'.
- Declare **int colors[6][6]** - Store each guess and its result.
- Write **print_board**(char history[][6], int colors[][5], int count) - you need to use the given **print_row()** function.
- Then play:
 - read validated guesses,
 - keep score,
 - check win,
 - print board.
 - On win print "Solved in N guesses!"
 - On loss reveal the word.

Guess 1/6: CRANE

[_]C [_]R [Y]A [_]N [G]E

- - - - -
- - - - -
- - - - -
- - - - -
- - - - -

Guess 2/6: FLAKE

[_]C [_]R [Y]A [_]N [G]E

[_]F [_]L [G]A [_]K [G]E

- - - - -
- - - - -
- - - - -
- - - - -

Guess 3/6: SCALE

[_]C [_]R [Y]A [_]N [G]E

[_]F [_]L [G]A [_]K [G]E

[G]S [G]C [G]A [G]L [G]E

- - - - -
- - - - -
- - - - -

Solved in 3 guesses!